

Software Integration Steps:

Step 1 – Create the Driver Header and Implementation Files

In this step, we establish the software foundation within the `pulp-runtime`. We create the necessary files to house the driver's API definitions and the underlying logic required to communicate with the hardware accelerator.

Files to Create

IN: `pulp-runtime`

File path 1:

`/pulp-runtime/include/wide_alu_driver.h`

File path 2:

`/pulp-runtime/drivers/wide_alu_driver.c`

Modification

Create the empty files in their respective directories within the `pulp-runtime` workspace.

Commands:

From the root of the `pulp-runtime` directory:

Bash

```
touch include/wide_alu_driver.h
touch drivers/wide_alu_driver.c
```

Notes

- **include/wide_alu_driver.h:** This header file will contain the function prototypes and external definitions. It acts as the interface for the `main.c` application.
- **drivers/wide_alu_driver.c:** This implementation file will contain the C code that performs memory-mapped I/O (MMIO) to control the accelerator.
- The `drivers/` directory may need to be created if it does not already exist in your specific runtime version.

Explanation

- `pulp-runtime` organizes software by separating interface definitions (`include/`) from functional logic (`drivers/` or `lib/`).
- By placing these files in the standard runtime structure:
 - The compiler can easily locate the header during the build process of the test application.
 - The build system (`pulp.mk`) can be configured to compile the driver logic into the final binary.
- This step only creates the physical files; no code is added, and the build system is not yet aware of them.

Step Summary

- Created the skeleton `wide_alu_driver.h` in the `include` directory.
- Created the skeleton `wide_alu_driver.c` in the `drivers` directory.
- Established the file-system foundation for the software driver integration.

Step 2 – Adding Driver Implementation and Header Content

In this step, the driver's logical structure is defined. The header file establishes the API while the implementation file contains the low-level functions required to write operands, configure operations, and poll for completion.

Files to Modify

IN: `pulp-runtime`

File path 1: `/pulp-runtime/include/wide_alu_driver.h`

File path 2: /pulp-runtime/drivers/wide_alu_driver.c

Modification

Populate the newly created files with the function declarations and their corresponding definitions.

wide_alu_driver.h Content:

```
#include <stdint.h>

#define WIDE_ALU_BASE_ADDR 0x1a400000

void set_op(uint8_t operation);

void set_delay(uint8_t delay);

void trigger_op(void);

void set_operands(uint32_t* a, uint32_t* b);

int poll_done(void);

void clear_error(void);

int wide_multiply(uint32_t a[32], uint32_t b[32], uint32_t result[64]);
```

wide_alu_driver.c Content as per repo:

```
#include <wide_alu_driver.h>

#include <stdint.h>

#include <stdio.h>

void set_delay(uint8_t delay)
{
    uint32_t volatile * ctrl2_reg = (uint32_t*)WIDE_ALU_CTRL2(0);
    uint32_t ctrl2_old_value;
    //Read old value
    ctrl2_old_value = *ctrl2_reg;
    //Overwrite operation bits
    *ctrl2_reg = ctrl2_old_value | (delay &
WIDE_ALU_CTRL2_DELAY_MASK)<<WIDE_ALU_CTRL2_DELAY_LSB;
}

void set_operands(uint32_t* a, uint32_t* b)
{

```

```

uint32_t volatile * op_a_reg_start = (uint32_t*)WIDE_ALU_OP_A_0(0);
uint32_t volatile * op_b_reg_start = (uint32_t*)WIDE_ALU_OP_B_0(0);
//Make sure we are in idle state before changing the operands
for (int i = 0; i<256/32; i++)
{
    op_a_reg_start[i] = a[i];
    op_b_reg_start[i] = b[i];
}
}
int poll_done(void)
{
    uint32_t volatile * status_reg = (uint32_t*)WIDE_ALU_STATUS(0);
    uint32_t current_status;
    do {
        current_status =
(*status_reg>>WIDE_ALU_STATUS_CODE_LSB) &WIDE_ALU_STATUS_CODE_MASK;
    } while(current_status == WIDE_ALU_STATUS_CODE_PENDING);
    if (current_status == WIDE_ALU_STATUS_CODE_IDLE)
        return 0;
    else
        return current_status;
}
void get_result(uint32_t* result)
{
    uint32_t volatile * result_reg_start = (uint32_t*)WIDE_ALU_RESULT_0(0);
    for (int i = 0; i<512/8; i++)
    {
        result[i] = result_reg_start[i];
    }
}
void clear_error(void)
{
    uint32_t volatile * ctrl1_reg = (uint32_t*)WIDE_ALU_CTRL1(0);
    //Trigger operation by writing to trigger bit

```

```

    *ctrl1_reg = (1 &
WIDE_ALU_CTRL1_CLEAR_ERR_MASK)<<WIDE_ALU_CTRL1_CLEAR_ERR_LSB;
}

```

A Potential Problem

There's a small but critical **index bug** in the `get_result` loop that may cause a memory corruption (buffer overflow).

The Bug: Result Loop Bounds

In `get_result`:

```
for (int i = 0; i < 512/8; i++)
```

- **512 / 8 = 64.**
- However, your `RESULT` multireg in the hardware (and your `wide_alu_regs.h`) only has **16 registers** (`WIDE_ALU_RESULT_MULTIREG_COUNT 16`).
- If you loop 64 times, the core will try to read 48 addresses past the end of your IP, which will likely cause a **Bus Error** or read garbage.

The Final Step for this File

Update your `wide_alu_driver.c` with the fixed bounds and the missing `set_op`, `trigger_op`, and `wide_multiply` functions required by your instructions.

the corrected and completed implementation:

```

#include <wide_alu_driver.h>
#include <stdint.h>
#include <stdio.h>

// Helper: Set the number of cycles for the operation
void set_delay(uint8_t delay) {
    uint32_t volatile * ctrl2_reg = (uint32_t*)WIDE_ALU_CTRL2(0);
    uint32_t ctrl2_old_value = *ctrl2_reg;
    // Clear old delay bits (mask is 0xFF, offset is 16)
    ctrl2_old_value &= ~(WIDE_ALU_CTRL2_DELAY_MASK <<
WIDE_ALU_CTRL2_DELAY_OFFSET);
    *ctrl2_reg = ctrl2_old_value | (delay & WIDE_ALU_CTRL2_DELAY_MASK) <<
WIDE_ALU_CTRL2_DELAY_OFFSET;
}

// Implement the required set_op function
void set_op(uint8_t operation) {
    uint32_t volatile * ctrl2_reg = (uint32_t*)WIDE_ALU_CTRL2(0);
    uint32_t ctrl2_old_value = *ctrl2_reg;
    // Overwrite the OPSEL bits (bits 2:0)
    ctrl2_old_value &= ~(WIDE_ALU_CTRL2_OPSEL_MASK <<
WIDE_ALU_CTRL2_OPSEL_OFFSET);

```

```

        *ctrl2_reg = ctrl2_old_value | (operation & WIDE_ALU_CTRL2_OPSEL_MASK)
<< WIDE_ALU_CTRL2_OPSEL_OFFSET;
    }

// Implement the required trigger_op function
void trigger_op(void) {
    uint32_t volatile * ctrl1_reg = (uint32_t*)WIDE_ALU_CTRL1(0);
    *ctrl1_reg = (1 << WIDE_ALU_CTRL1_TRIGGER_BIT);
}

void set_operands(uint32_t* a, uint32_t* b) {
    uint32_t volatile * op_a_reg_start = (uint32_t*)WIDE_ALU_OP_A_0(0);
    uint32_t volatile * op_b_reg_start = (uint32_t*)WIDE_ALU_OP_B_0(0);
    for (int i = 0; i < 8; i++) { // 256 bits / 32 bits = 8 regs
        op_a_reg_start[i] = a[i];
        op_b_reg_start[i] = b[i];
    }
}

int poll_done(void) {
    uint32_t volatile * status_reg = (uint32_t*)WIDE_ALU_STATUS(0);
    uint32_t current_status;
    do {
        current_status = (*status_reg >> WIDE_ALU_STATUS_CODE_OFFSET) &
WIDE_ALU_STATUS_CODE_MASK;
    } while(current_status == WIDE_ALU_STATUS_CODE_VALUE_PENDING);

    return (current_status == WIDE_ALU_STATUS_CODE_VALUE_IDLE) ? 0 :
current_status;
}

void get_result(uint32_t* result) {
    uint32_t volatile * result_reg_start =
(uint32_t*)WIDE_ALU_RESULT_0(0);
    for (int i = 0; i < 16; i++) { // FIXED: 512 bits / 32 bits = 16 regs
        result[i] = result_reg_start[i];
    }
}

// The main wrapper function required by your assignment
int wide_multiply(uint32_t a[8], uint32_t b[8], uint32_t result[16]) {
    set_operands(a, b);
    set_op(WIDE_ALU_CTRL2_OPSEL_VALUE_MUL);
    trigger_op();
    int status = poll_done();
    if (status == 0) {
        get_result(result);
    }
    return status;
}

```

Notes

- **Address Mapping:** The driver relies on macros like `WIDE_ALU_OP_A_0(0)` which must be defined in the autogenerated register header.
- **Operand Width:** The `set_operands` function handles two 256-bit inputs by iterating through eight 32-bit register writes.
- **Result Width:** The `get_result` function must be implemented to read back the 512-bit result (16 registers).

Explanation

- The **Header File** provides a contract for the application, abstracting the hardware details behind clear function names.
- The **Implementation File** performs the actual "work" by:
 - Converting 32-bit software arrays into the wide data types expected by the hardware.
 - Managing the control flow, such as ensuring the accelerator is in an `IDLE` state before a new operation begins.
- This step provides the logic but does not yet link it to the actual hardware base address or the build system.

Step Summary

- Populated `wide_alu_driver.h` with the API required to utilize the custom accelerator.
- Implemented the register-access logic in `wide_alu_driver.c`.
- Prepared the software stack to handle 256-bit operands and 512-bit results.

Step 3 – Generating the Register Map Header

In this step, we use the `regtool` utility to bridge the gap between the hardware description and the software driver. This tool automates the creation of a C header file that defines the exact memory offsets and bitfields required to control the Wide ALU.

Files Involved

Input File: `$HOME/GP/rtl/wide_alu/wide_alu_top.sv/wide_alu.hjson`

Output File: `$HOME/GP/pulp-runtime/include/wide_alu_regs.h`

Tool Location:

`/home/icpedia/pulp/ips/register_interface/vendor/lowrisc_opentitan/util/regtool.py`

Modification

Run the `regtool.py` (`reggen`) script with the `C-defines` flag to generate the register definition header.

Command Adjust to your paths:

```
python3 $REGGEN -D rtl/wide_alu/wide_alu.hjson > pulp-  
runtime/include/wide_alu_regs.h
```

Notes

- **Tool Origin:** The `regtool.py` is sourced from the `register_interface` IP because it is specifically patched to support the Generic Register Interface Protocol used in this exercise.
- **-D Flag:** This flag instructs the tool to render C-style `#define` statements instead of SystemVerilog RTL or JSON.
- **Schema Compatibility:** The tool reads the `wide_alu.hjson` to determine register names (e.g., `CTRL1`, `OP_A`), access permissions (e.g., `wo`, `ro`), and bit widths.

Explanation

- The generated file, `wide_alu_regs.h`, acts as a "map" for the software driver.
- It provides symbolic constants for:
 - **Register Offsets:** Such as `WIDE_ALU_OP_A_0_REG_OFFSET` (0x0) and `WIDE_ALU_STATUS_REG_OFFSET` (0x88).
 - **Bitfield Masks:** Allowing the software to isolate specific bits like the `TRIGGER` bit or the `OPSEL` field.
- Using this generated header prevents manual errors that occur when hardcoding addresses and ensures that software is always synchronized with the hardware RTL.

Step Summary

- Located the compatible `regtool.py` utility within the PULP IP directory.
- Successfully generated `wide_alu_regs.h` containing all necessary C definitions.
- Provided the driver with a type-safe and symbolic way to access the Wide ALU peripheral registers.

Step 4 – Patching the Runtime Environment

This step integrates the newly created driver files into the `pulp-runtime` build system. By modifying the master header and the build rules, the Wide ALU driver becomes a permanent part of the SDK, allowing any application to access the accelerator's functionality.

Files to Modify

1. Master Header File

- **Repository:** `pulp-runtime`
- **File path:** `/pulp-runtime/include/pulp.h`

2. Build System Rules

- **Repository:** `pulp-runtime`
- **File path:** `/pulp-runtime/rules/pulpos/targets/pulp.mk`

Modification

1. Update `pulp.h` Add the following include directives to the end of the file (before the final `#endif`) to expose the driver API and register definitions to the user application:

```
#include "wide_alu_regs.h"  
#include "wide_alu_driver.h"
```

2. Update `pulp.mk`

Locate the source file list (usually `PULP_SDK_SRCS`) and append the driver implementation file to ensure it is compiled during the build process:

Makefile

```
PULP_SRCS      += drivers/wide_alu_driver.c
```

Notes

- **Visibility:** Including `wide_alu_driver.h` in `pulp.h` ensures that the user only needs to include `#include <pulp.h>` in their `main.c` to access the accelerator.
- **Compilation:** Adding the `.c` file to `pulp.mk` ensures the linker can resolve the function symbols for `set_op` and `wide_multiply`.
- **Order:** Ensure `wide_alu_regs.h` is included before or alongside the driver header, as the driver logic depends on the generated macros.

Explanation

- `pulp.h` acts as the primary entry point for the PULPissimo software stack; patching it makes the custom IP a "first-class" peripheral.
- The `pulp.mk` file governs the compilation of the entire runtime library; without this modification, the functions defined in the driver would result in "undefined reference" errors during the linking stage of the test application.
- This step finalizes the environment setup, moving the project from the "configuration" phase to the "implementation" phase.

Step Summary

- Exposed the Wide ALU API and register map in the master SDK header (`pulp.h`).
- Registered the driver source file in the global build rules (`pulp.mk`).
- Successfully linked the software stack to the hardware-specific register definitions.

Step 5 – Initializing the Driver Source Code

In this step, we begin the actual coding phase by initializing the implementation file with its required dependencies. This ensures that the driver has access to the PULP system environment, the register map, and the correct hardware base address.

File to Modify

Repository: `pulp-runtime`

File path: `/pulp-runtime/drivers/wide_alu_driver.c`

Modification

Add the core include directives and the hardware base address definition to the top of the file.

Code to Add:

C

```
#include "pulp.h"
#include "wide_alu_driver.h"
#include "wide_alu_regs.h"

// Define the base address for the hardware
#define WIDE_ALU_BASE 0x1A400000
```

Notes

- **pulp.h:** Provides access to basic SoC functions and hardware abstraction layers.
- **wide_alu_driver.h:** Ensures the implementation matches the function prototypes defined for the application.
- **wide_alu_regs.h:** Provides the symbolic names for register offsets generated before
- **WIDE_ALU_BASE:** This constant map the driver to the specific memory region (0x1A40_0000) where the accelerator was integrated into the SoC AXI crossbar.

Explanation

- These lines act as the "preamble" for the driver.
- Without including `wide_alu_regs.h`, the driver would not recognize register names like `CTRL1` or `OP_A`.
- Defining the `WIDE_ALU_BASE` here creates a single point of truth for the IP's location in the memory map, making the code easier to maintain if the hardware address changes.

Step Summary

- Initialized the implementation file with necessary system and local headers.

- Defined the memory-mapped base address for the Wide ALU IP.
- Prepared the source file for the implementation of arithmetic and control functions.

Step 6 – Defining the Peripheral Base Address in the SoC Map

In this step, we ensure the hardware base address is recognized by the SoC's architectural definitions. While the driver uses this address locally, adding it to the chip's central memory map ensures consistency across the entire `pulp-runtime` and prevents address conflicts.

Files to Modify

Repository: `pulp-runtime`

File path: `/pulp-runtime/include/archi/chips/pulp/memory_map.h`

Modification

Add the Wide ALU base address to the SoC memory map header. This file defines where all peripherals (like UART, GPIO, and now your Accelerator) live in the 32-bit address space.

Code to Add:

C

```
/* new axi ip */
#define WIDE_ALU0_BASE_ADDR 0x1a400000
```

Notes

- **Address Consistency:** The address `0x1a400000` must match the hardware address decoding rules defined during the AXI crossbar integration.
- **archi Directory:** Files under `archi/chips/pulp/` are low-level architectural headers that describe the specific "flavor" of the PULPissimo chip you are simulating.
- **Prefixing:** Using a prefix like `WIDE_ALU0_` follows the PULP naming convention for peripherals.

Explanation

- **Centralization:** By adding the address to `memory_map.h`, you allow other parts of the system (like the kernel or other drivers) to know where the Wide ALU is located without re-defining it in every file.
- **Conflict Prevention:** Seeing all addresses in one file helps you verify that `0x1a400000` does not overlap with existing regions like L2 memory or the APB peripherals.

- **Driver Integration:** Now, instead of hardcoding `0x1a400000` inside your `.c` file, you can simply use the constant `WIDE_ALU0_BASE_ADDR`.

Step Summary

- Updated the architectural memory map header `memory_map.h`.
- Defined `WIDE_ALU0_BASE_ADDR` as `0x1a400000`.
- Synchronized the software's view of the SoC address space with the hardware's AXI decoding logic.

Step 7 – Validating the Stack with the Test Application

In this final software step, we create the test application (`main.c`) to validate the entire hardware/software stack. This application utilizes the driver API to communicate with the integrated AXI peripheral and verify that the system correctly computes the expected result.

File to Create

First run these commands

```
cd ~/GP/examples
```

```
mkdir axi_ip
```

```
touch main.c
```

File path: `~/GP/examples/axi_ip/main.c`

Modification

Create the `main.c` file and populate it with the test logic to multiply two values using the hardware accelerator.

Code Content:

```
#include <stdio.h>
#include "pulp.h"

int main()
{
    printf("Hello World!\n");

    uint32_t a[8];          // 256 bits total [cite: 20]
    uint32_t b[8];          // 256 bits total [cite: 20]
    uint32_t result[16];    // 512 bits total [cite: 20]

    // Initialize memory
```

```

memset(a, 0, sizeof(a));
memset(b, 0, sizeof(b));
memset(result, 0, sizeof(result));

// Set test operands
a[0] = 3;
b[0] = 5;

// Configure hardware delay factor
set_delay(50);

// Execute hardware-accelerated multiplication [cite: 49, 52]
wide_multiply(a, b, result);

// Output result to UART
printf("Result [0]: %d\n", result[0]);

return 0;
}

```

Notes

- **Operand Sizing:** While the original code used larger arrays, the `wide_alu` hardware specifically expects **8 words (256 bits)** for operands and provides **16 words (512 bits)** for results.
- **Delay Parameter:** The `set_delay(50)` call tests the "deacceleration factor" logic within the IP wrapper.
- **Standard Output:** Successful execution will result in the string `Result [0]: 15` appearing on your terminal.

Explanation

- **pulp.h Inclusion:** This header provides the prototypes for your driver functions because of the patch applied in **Step 5.4**.
- **MMIO Orchestration:** The call to `wide_multiply` triggers a chain of events:
 1. Software writes to the AXI Slave.
 2. The protocol converter updates the Register File.
 3. The Hardware Accelerator processes the data.
- **Verification:** This test confirms that AXI address decoding, register-file wiring, and software driver logic are all correctly synchronized.

Step Summary

- Created the `main.c` validation application.
- Configured test operands (3 and 5) and a deacceleration factor.
- Verified the end-to-end functionality of the custom AXI IP integration.

Step 8 – Configuring the Application Makefile

In this final step, we configure the local `Makefile` for the test application. This file tells the PULP SDK which source files to compile for the Fabric Controller (FC) and which optimization levels to use.

File to Modify

File to Create

First run these commands

```
touch Makefile
```

File path: `~/GP/examples/axi_ip/ Makefile`

Modification

Define the application name and source files, and include the standard PULP runtime rules.

Code Content:

Makefile

```
PULP_APP = main
PULP_APP_FC_SRCS = main.c
PULP_APP_HOST_SRCS = main.c
PULP_CFLAGS = -O3 -g

include $(PULP_SDK_HOME)/install/rules/pulp_rt.mk
```

Notes

- **pulp_rt.mk:** No, you do not modify this file. It is a "read-only" system file provided by the SDK that contains the standard compilation and simulation recipes (like `make clean` `all` `run`).
- **PULP_APP_FC_SRCS:** This variable specifies that `main.c` should be compiled for the Fabric Controller, which is the main RISC-V core in PULPissimo.
- **PULP_CFLAGS:** The `-O3` flag enables high-level optimizations, while `-g` includes debug symbols for simulation.

Explanation

- **Inclusion vs. Modification:** By *including* `pulp_rt.mk` instead of *modifying* it, you keep your project portable. The system-wide changes you made earlier in `pulp.mk` (Step 5.4) will automatically be pulled in by this include statement.

- **SDK Integration:** This Makefile acts as the bridge between your custom code and the entire PULP toolchain (compiler, linker, and simulator).
- **Execution:** Once this file is saved, you can run the command `make clean all run` to trigger the compilation of both your `main.c` and the `wide_alu_driver.c` we integrated into the SDK.

Step Summary

- Created the application-level `Makefile`.
- Pointed the build system to `main.c`.
- Linked the application to the standard PULP runtime rules without modifying core SDK files.

Step 9 – Final fixes

1. Fix `pulp-runtime/include/wide_alu_driver.h`

2. Fix `pulp-runtime/drivers/wide_alu_driver.c`

This file needs to be updated to use the **Offset Macros** correctly (adding them to the Base Address).

Please download the both of them from the following drive link

`wide_alu_driver.h` , `wide_alu_driver.c`

<https://drive.google.com/drive/folders/1iVXoUbCY0rCGpfaMywgJEAmJNe6lE7FX>

Step 10 – Final result

Finally you should got this result after running the test

```
cd sim
```

```
make build
```

```
vopt +acc=npr -o vopt_tb tb_pulp -floatparameters+tb_pulp -work work
```

```
cd examples/axi_ip
```

make clean all run

```
# [TB] 8675401ns retrying debug reg access
# [TB] 8704701ns retrying debug reg access
# [TB] 8734001ns retrying debug reg access
# [TB] 8763301ns retrying debug reg access
# [TB] 8792601ns retrying debug reg access
# [TB] 9351301ns - Waiting for end of computation
# [STDOUT-CL31_PE0] Hello World!
# [STDOUT-CL31_PE0] Result [0]: 15
# [TB] 10010001ns - Received status core: 0x00000000
# ** Note: $stop : /home/marwan/GP/rtl/tb/tb_pulp.sv(1008)
# Time: 10010001 ns Iteration: 0 Instance: /tb_pulp
# Break at /home/marwan/GP/rtl/tb/tb_pulp.sv line 1008
# Stopped at /home/marwan/GP/rtl/tb/tb_pulp.sv line 1008
# End time: 23:31:41 on Feb 08,2026, Elapsed time: 0:01:31
```

Appendix (To be removed)

These two files (`wide_alu.sv` and `wide_alu_pkg.sv`) are handwritten reference RTL, NOT generated.

They are **provided by the training authors** as a *golden example accelerator*.

- Not generated by reggen
- Not autogenerated
- Intentionally written by a human
- Meant to be reused as-is in this exercise

What is generated vs handwritten

1 Generated by reggen (tool output)

These come from `wide_alu.hjson`:

- `wide_alu_reg_pkg.sv`
- `wide_alu_reg_top.sv`

These files:

- Define register structs (`*_reg2hw_t`, `*_hw2reg_t`)
- Implement the register file logic
- Handle SW/HW access rules (`hwext`, `hwqe`, etc.)

You **never** write these by hand.

2 Handwritten accelerator RTL

These are **NOT** generated:

wide_alu_pkg.sv

```
package wide_alu_pkg;
    typedef enum logic[2:0] { ADD, SUB, MUL, XOR, AND, OR } optype_e;
    typedef enum logic[1:0] { IDLE, PENDING, ERROR_WRITE, ERROR_OPCODE }
status_e;
endpackage
```

Purpose:

- Defines **semantic meaning** of operations and states
- Used by:
 - wide_alu.sv
 - wide_alu_top.sv
 - regfile wiring (casts, enums)

This is **domain logic**, not infrastructure → must be handwritten.

wide_alu.sv

This is the **actual accelerator**.

It:

- Implements the 256-bit datapath
- Handles:
 - Trigger semantics
 - Deacceleration delay
 - Error handling
 - Status reporting
- Matches the **exact register semantics** defined in HJSON

This is **the IP you are integrating**, not something reggen can infer.

Why reggen can't generate this

Reggen only knows:

- Registers
- Fields
- Access permissions

It **does NOT** know:

- What ADD/MUL/XOR actually mean
- How to pipeline operations
- How long an operation takes

- How status transitions work

That logic **must be written manually**.

Nothing is conceptually missing.

Key takeaway (this is gold)

Reggen generates the “register plumbing”, not the IP.
The IP itself is always handwritten.

Conceptual Explanation

You are connecting **three different worlds**:

1. **AXI** (used by the core / SoC)
2. **REG_BUS interface** (used by the AXI→reg protocol converter)
3. **reggen-generated register file**, which **does NOT use interfaces**, but **plain structs**

The problem is:

- `axi_to_reg_intf` outputs a **SystemVerilog interface** (`REG_BUS`)
- `wide_alu_reg_top` expects **structs** (`reg_req_t`, `reg_rsp_t`)

So we need: a **lossless adapter** between an *interface* and *structs*

That is exactly what these macros do.

What each part does

1. Struct type declarations

```
`REG_BUS_TYPEDEF_REQ(reg_req_t, addr_t, data_t, strb_t)
`REG_BUS_TYPEDEF_RSP(reg_rsp_t, data_t)
```

These macros:

- **define two packed structs**
- matching the Generic Register Interface protocol

They expand into something conceptually like:

```
typedef struct packed {
    addr_t addr;
    logic write;
    data_t wdata;
}
```

```

    strb_t wstrb;
    logic valid;
} reg_req_t;

typedef struct packed {
    data_t rdata;
    logic error;
    logic ready;
} reg_rsp_t;

```

So now we have **strongly-typed structs** for requests and responses.

2. Declaring the struct wires

```

reg_req_t to_reg_file_req;
reg_rsp_t from_reg_file_rsp;

```

These are the **actual wires** that will connect:

- AXI $\rightarrow$ regfile (to_reg_file_req)
- regfile $\rightarrow$ AXI (from_reg_file_rsp)

3. Converting interface $\rightleftharpoons$ struct

```

`REG_BUS_ASSIGN_TO_REQ(to_reg_file_req, axi_to_regfile)
`REG_BUS_ASSIGN_FROM_RSP(axi_to_regfile, from_reg_file_rsp)

```

These macros:

- generate **continuous assignments**
- field-by-field
- between an **interface** and a **struct**

From the macro source you pasted, this expands to logic like:

```

assign to_reg_file_req = '{
    addr  : axi_to_regfile.addr,
    write : axi_to_regfile.write,
    wdata : axi_to_regfile.wdata,
    wstrb : axi_to_regfile.wstrb,
    valid : axi_to_regfile.valid
};

assign axi_to_regfile.rdata = from_reg_file_rsp.rdata;
assign axi_to_regfile.error = from_reg_file_rsp.error;
assign axi_to_regfile.ready = from_reg_file_rsp.ready;

```

So:

- **Interface $\rightarrow$ struct** for requests

- **Struct** → **interface** for responses

No logic, no latency, no behavior — just **signal mapping**.

Why this exists

- Many EDA tools struggle with interfaces inside autogenerated code
- reggen intentionally uses **plain structs**
- ETH's `register_interface` library provides **clean macros** to bridge both worlds
- This keeps autogenerated RTL **tool-agnostic and robust**

This is **not glue code** — it's a **design pattern**.